



Chapter one

Introduction

To

Computer organization and Architecture

Outline

- Logic gates
- Boolean Algebra
- Combinational circuit
- Flip Flops
- Sequential circuits

Introduction

❖ *What is digital system ?*

- ❖ It is designed to store, process, and communicate information in digital form.
- ❖ They are found in a wide range of applications, including process control, communication systems, digital instruments, E.g. computer
- ❖ A computer manipulates information in digital, or in binary form.
- ❖ A binary number has only two discrete values — **zero** or **one**.
- ❖ Each of these discrete values is represented by the OFF and ON status of an electronic switch
- ❖ All computers understand only binary numbers.

Introduction

What is Computer Architecture ?

- ❖ It is concerned with the way *hardware components are connected together* to form a computer system.
- ❖ Computer Architecture helps us to understand the functionalities of a system
- ❖ A programmer can view architecture in terms of instructions, addressing modes and register.
- ❖ It involves Logic (Instruction sets, Addressing modes, Data types, Cache optimization)

Introduction

What is Computer organization?

- ❖ it is concerned with the structure of a computer system as seen by the user.
- ❖ It deals with the connection of components in a system.
- ❖ It tells us *how exactly all the units in the system are arranged and interconnected.*
- ❖ It expresses the realization of architecture.
- ❖ An organization is done on the basis of architecture.
- ❖ It deals with low-level design issues.

Boolean Algebra

- ❖ **Boolean algebra:-** is algebra for the manipulation of objects that can take on only **two** values, typically true and false.
 - ✓ It is common to interpret the digital value **0** as false and the digital value **1** as true.
- ❖ **Boolean Expression:** -it combination of the variables and operation
- ❖ **Boolean Function:** -A Boolean function typically has one or more input values and yields **a result**.
- ❖ A Boolean operator can be completely described using a **table** that list inputs, *all possible values for these inputs*, and the resulting values of the operation.

Cont..

- ❖ A **truth table**, in tabular form that shows the **relationship**, between the input values and the result of a specific Boolean operator.
 - ✓ e.g. of Boolean operator AND, OR, NOT etc.
- ❖ The AND operator is also known as a **Boolean product**.
- ❖ The Boolean expression xy is equivalent to the expression $x * y$ and is read “x and y.”
- ❖ The behavior of AND operator is characterized by the truth table shown in Table below

Inputs		Outputs
x	y	xy
0	0	0
0	1	0
1	0	0
1	1	1

Cont..

- ❖ The OR operator is often referred to as a **Boolean sum**.
- ❖ The expression $x+y$ is read “x or y”. The truth table for OR is shown in the following table

Inputs		Outputs
x	y	$x+y$
0	0	0
0	1	1
1	0	1
1	1	1

- ❖ NOT operator :- x' is read as “NOT x.” The truth table for NOT is shown in x

Inputs	Outputs
x	$\bar{x}$
0	1
1	0

Cont..

- ❖ **DeMorgan's** law provides an easy way of finding the complement of a Boolean function.

$$\overline{(xy)} = \bar{x} + \bar{y} \quad \text{and} \quad \overline{(x+y)} = \bar{x}\bar{y}$$

x	y	(xy)	$\overline{(xy)}$	$\bar{x}$	$\bar{y}$	$\bar{x} + \bar{y}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Cont..

❖ Complement

x	y	z	$y\bar{z}$	$\bar{x}+y\bar{z}$	$\bar{y}+z$	$x(\bar{y}+z)$
0	0	0	0	1	1	0
0	0	1	0	1	1	0
0	1	0	1	1	0	0
0	1	1	0	1	1	0
1	0	0	0	0	1	1
1	0	1	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	0	1	1

Cont..

- ❖ The rule of precedence for Boolean operators give **NOT** top priority, followed by **AND**, and then **OR**
- ❖ **Boolean function $F=X+Y'Z$ Truth table**

Inputs				Outputs
x	y	z	$\bar{y}$ $\bar{y}z$	$x + \bar{y}z = F$
0	0	0	1 0	0
0	0	1	1 1	1
0	1	0	0 0	0
0	1	1	0 0	0
1	0	0	1 0	1
1	0	1	1 1	1
1	1	0	0 0	1
1	1	1	0 0	1

Cont..

❖ Boolean Identities

- Boolean expression can be simplified, but we need new **identities**, or **laws**, that apply to Boolean algebra instead of regular algebra.

1	$A + A = A$	Idempotent Law for OR
2	$A * A = A$	Idempotent Law for AND
3	$A + B = B + A$	Commutative Law for OR
4	$A * B = B * A$	Commutative Law for AND
5	$A + (B + C) = (A + B) + C$	Associative Law for OR
6	$A * (B * C) = (A * B) * C$	Associative Law for AND
7	$A * (B + C) = A * B + A * C$	Distributive Law for AND over OR
8	$A + B * C = (A + B) * (A + C)$	Distributive Law for OR over AND
9	$A + 1 = 1$	Law of Union
10	$A * 0 = 0$	Law of Intersection
11	$A * (A + B) = A$	Law of Absorption
12	$A + A * B = A$	Law of Absorption
13	$A * 1 = A$	Identity Law for AND
14	$A + 0 = A$	Identity Law for OR

Cont..

15	$\overline{\overline{A}} = A$	Double Negative Law
16	$A + \overline{A} = 1$	Law of Complement for OR
17	$A * \overline{A} = 0$	Law of Complement for AND
18	$\overline{A + B} = \overline{A} * \overline{B}$	DeMorgan's Law
19	$\overline{A * B} = \overline{A} + \overline{B}$	DeMorgan's Law
20	$A \oplus B = A * \overline{B} + \overline{A} * B = \overline{A} * B + A * \overline{B}$	Exclusive OR (XOR)
21	$\overline{A \oplus B} = A * B + \overline{A} * \overline{B}$	Exclusive NOR (XNOR)
22	$A + \overline{A} * B = A + B$	Law of the "disappearing opposite"
23	$(A + B) * (A + C) = A + B * C$	Reverse of Law #8
24	$(A + B) * (C + D) = A * C + A * D + B * C + B * D$	FOIL (First,Outer,Inner,Last) Distribut

Note: **AND** will always be expressed explicitly with the * operator

Cont..

❑ Simplification of Boolean Expressions

- ❖ The algebraic identities we studied in algebra class allow us to reduce algebraic expression to their **simplest** form.
- ❖ To prove the equality of two Boolean expressions, you can also create the truth tables for each and compare. If the truth tables are **identical**, the expressions are **equal**
- ❖ E.g. $(x+y)(x'+y)$

Proof	Identity Name
$(x+y)(\bar{x}+y) = x\bar{x}+xy+y\bar{x}+yy$	Distributive Law
$= 0+xy+y\bar{x}+yy$	Inverse Law
$= 0+xy+y\bar{x}+y$	Idempotent Law
$= xy+y\bar{x}+y$	Identity Law
$= y(x+\bar{x})+y$	Distributive Law (and Commutative Law)
$= y(1)+y$	Inverse Law
$= y+y$	Identity Law
$= y$	Idempotent Law

Cont..

Exercise

A, show that Simplify: $\bar{A}(A + B) + (B + AA)(A + \bar{B})$: $= A + B$

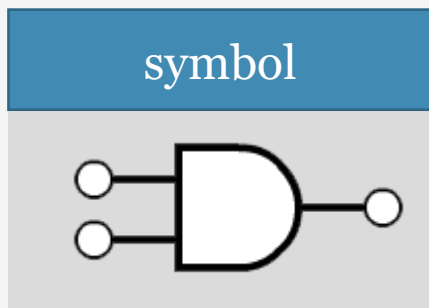
B Simplify: $\overline{AB}(\bar{A} + B)(\bar{B} + B)$: $= A'$

Logic gates

- ❖ Logic gates are the basic building blocks of any digital system.
- ❖ It is an electronic circuit having one or more than one input and only one output.
- ❖ The relationship between the input and the output is based on a certain logic.
- ❖ Based on this, logic gates are named as AND gate, OR gate, NOT gate etc.

AND LOGIC GATE

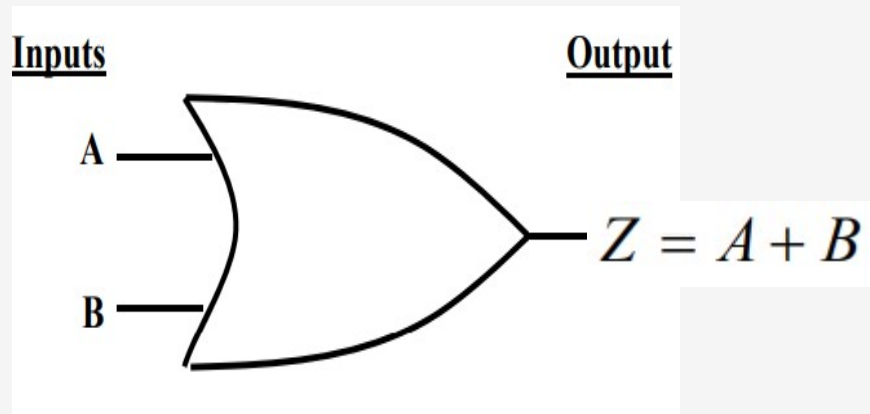
- ❖ An AND gate is a digital logic gate with two or more inputs and one output that performs logical conjunction.
- ❖ The output of an AND gate is true only when all of the inputs are true.
- ❖ If one or more of an AND gate's inputs are false, then the output of the AND gate is false $C = A.B$



Truth table		
A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

OR Logic Gate

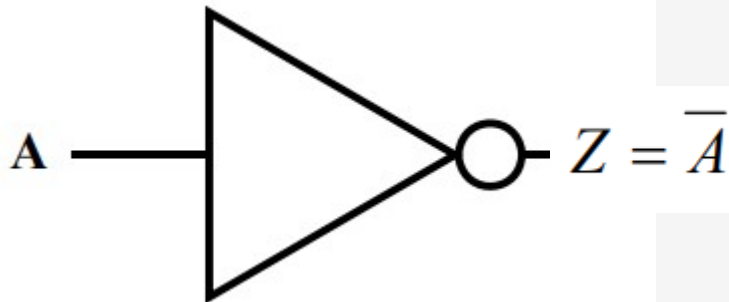
- ❖ The OR gates has two or more input signals but only one output signal.
- ❖ If **any** input signal is high, the output signal is high.
- ❖ This can be shown in a table below with all the possible inputs and corresponding outputs.



<u>Inputs</u>		<u>Output</u>
A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

NOT or Inverter

- ❖ A Low input produces a high output, and a high input produces low output.
- ❖ In binary format if the input is 0 the output will be 1 and if the input is 1 then the output will be 0.
- ❖ The table shows the input and output possibilities.
- ❖ Logical Function

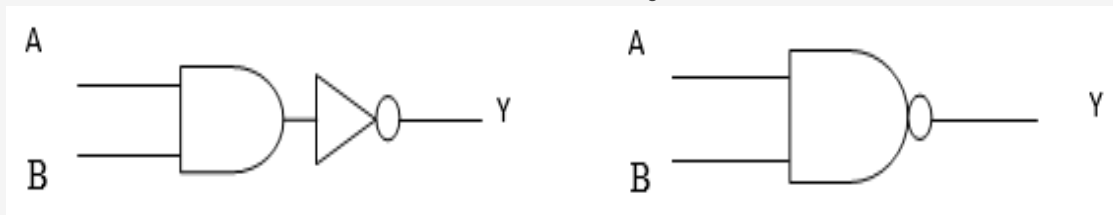


<u>Input</u>	<u>Output</u>
A	Z
0	1
1	0

NAND Gate- Cont'd

- ❖ NAND Gate is a combination of an AND gate with an inverter.

An AND Gate followed by an inverter.



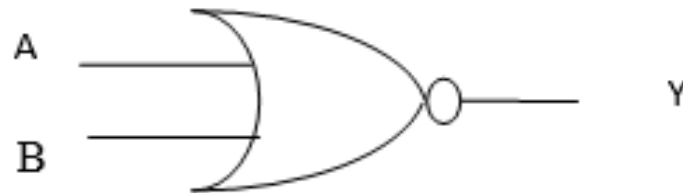
- ❖ Whatever the output of the AND gate, **it will be inverted by the inverter.**
- ❖ The sign and the table is shown below.
- ❖ The NAND operation is called Universal Operation or gate.

$$Y = \overline{AB}$$

NAND Sign

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

- ❖ NOR Gate is a combination of an OR gate with an inverter.
- An NOR Gate followed by an inverter.



$$Y = \overline{A+B}$$

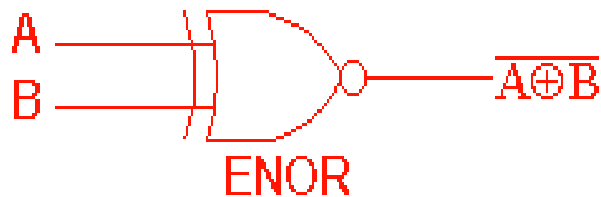
NOR Sign

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

- ❖ **Reading assignment XOR**

Exclusive NOR or XNOR Gate- Cont'd

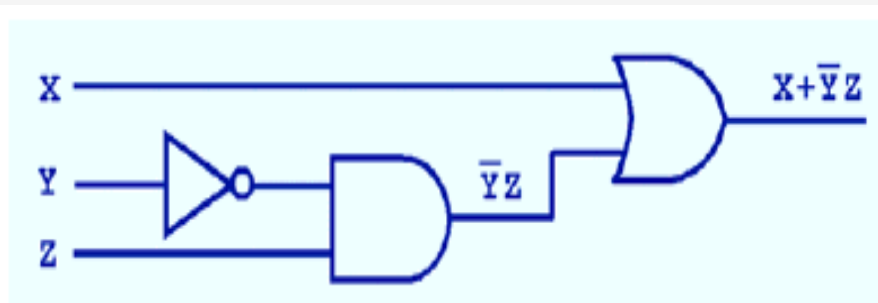
- ❖ it is abbreviated as XNOR. This is logically equivalent to an XOR gate followed by an inverter.
- ❖ However, an output “1” is only obtained if **BOTH** of its inputs are at the same logic level, either binary “1” or “0”. For example, “00” or “11”. This input combination would then give us the Boolean expression of:



2 Input EXNOR gate		
A	B	$\overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1

Cont..

- ❖ Every computer is built using collections of **gates** that are all connected by way of wires acting as signal gateway.
- ❖ More complex Boolean expressions can be represented as combinations of AND, OR, and NOT gates, resulting in a logic diagram that describes the entire expression



$$F(X, Y, Z) = X + \bar{Y}Z$$

lecture 2

Karnaugh Maps

- ❖ In many digital circuits and practical problems we need to find expression with minimum variables.
- ❖ We can minimize Boolean expressions of 3, 4 variables very easily using K-map without using any Boolean algebra theorems.
- ❖ K-map can take two forms Sum of Product (SOP) and Product of Sum (POS) according to the need of problem.
- ❖ K-map is table like representation but it gives more information than TRUTH TABLE.
- ❖ We fill grid of K-map with 0's and 1's then solve it by making groups.
- ❖ Each combination of the variables in a truth table is called a minterm.

Cont..

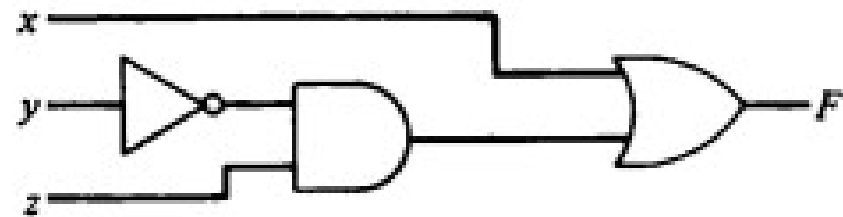
Steps to solve expression using K-map-

- 1) Select K-map according to the number of variables.
- 2) Identify minterms or maxterms as given in problem.
- 3) For SOP put 1's in blocks of K-map respective to the minterms (0's elsewhere).
- 4) For POS put 0's in blocks of K-map respective to the maxterms(1's elsewhere).
- 5) Make rectangular groups containing total terms in power of two like 2,4,8 ..(except 1) and try to cover as many elements as you can in one group.
- 6) From the groups made in step 5 find the product terms and sum them up for SOP form

Figure 1-3 Truth table and logic diagram for $F = x + y'z$.

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

(a) Truth table



minterm

(b) Logic diagram

The information contained in a truth table may be expressed in compact form by listing the decimal equivalent of those minterms that produce a 1 for the function.

Example: the compact form the truth table above is:

$$F(x, y, z) = \sum (1, 4, 5, 6, 7)$$

The letters in parentheses list the binary variables in the order that they appear in the truth table.

The symbol Σ stands for the sum of the minterms that follow in parentheses.

The minterms that produce 1 for the function are listed in their decimal equivalent. The minterms missing from the list are the ones that produce 0 for the function.

The maps for functions of two, three, and four variables are shown in Fig.

		B	
		0	1
A	0	0	1
	1	2	3

Two-variable

		B			
		00	01	11	10
A	0	0	1	3	2
	1	4	5	7	6

Three-variable

		C			
		00	01	11	10
A	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

Four-variable

The number of squares in a map of n variables is 2^n . The 2^n minterms are listed by an equivalent decimal number for easy reference. The minterm numbers are assigned in an orderly arrangement such that adjacent squares represent minterms that differ by only one variable.

adjacent squares

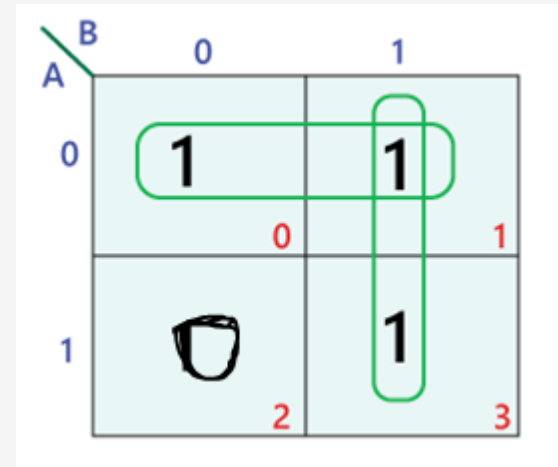
- The adjacent squares in the map are adjacent
- The squares at the extreme ends of the same horizontal row are also adjacent
- The same applies to the top and bottom squares of a column.
- As a result, the four corner squares of a map must also be considered to be adjacent.

-
- A Boolean function represented by a truth table is plotted into the map by inserting 1's in those squares where the function is 1.
 - The squares containing 1's are combined in groups of adjacent squares. These groups must contain a number of squares that is an integral power of 2.
 - Groups of combined adjacent squares may share one or more squares with one or more groups.
 - Each group of squares represents an algebraic term, and the OR of those terms gives the simplified algebraic expression for the function.

Example

Example 1: $Y = A'B' + A'B + AB$

$$y = \sum_{A,B} (0,1,3)$$



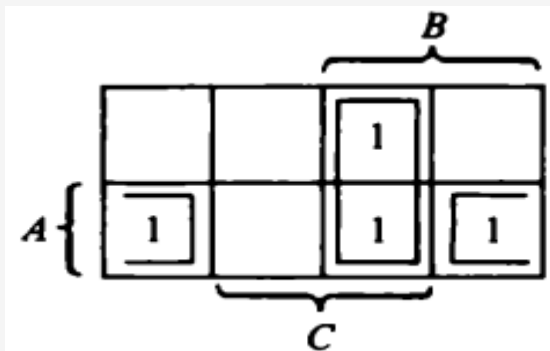
Simplified expression: $Y = A' + B$

Exercise: simplify the following function using map method

- $y = a'b' + ab' = b'$
- $y = a'b + ab' + ab = a + b$

three variable example 1

$$F(A, B, C) = \sum (3, 4, 6, 7)$$

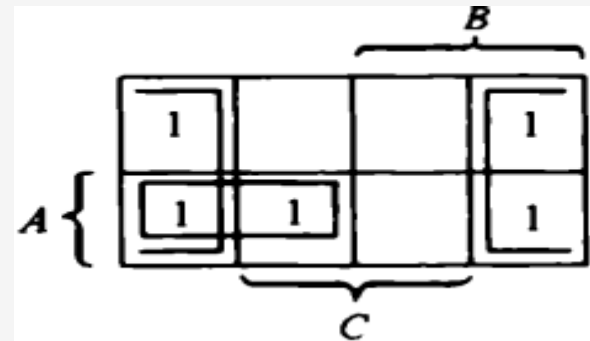


$$F = BC + AC'$$

three variable example 2

$$F(A, B, C) = \sum (0, 2, 4, 5, 6)$$

The map of the function



Map for $F(A, B, C) = \sum (0, 2, 4, 5, 6)$.

the result is

$$F = C' + AB'$$

Exercise for three variable

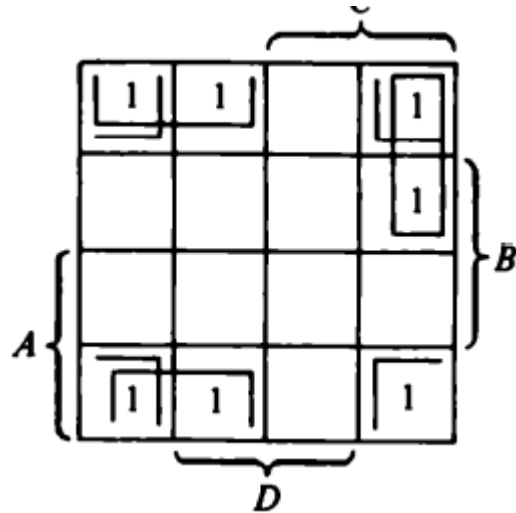
- **exercise:-** $Z = WX'Y' + WY + W'YZ$
- $Y = abc' + abc = ab$

Four variable example

$$F(A, B, C, D) = \sum (0, 1, 2, 6, 8, 9, 10)$$

Map of

$$F(A, B, C, D) = \sum (0, 1, 2, 6, 8, 9, 10)$$



The final result is

$$F = B'D' + B'C' + A'CD'$$

Cont..

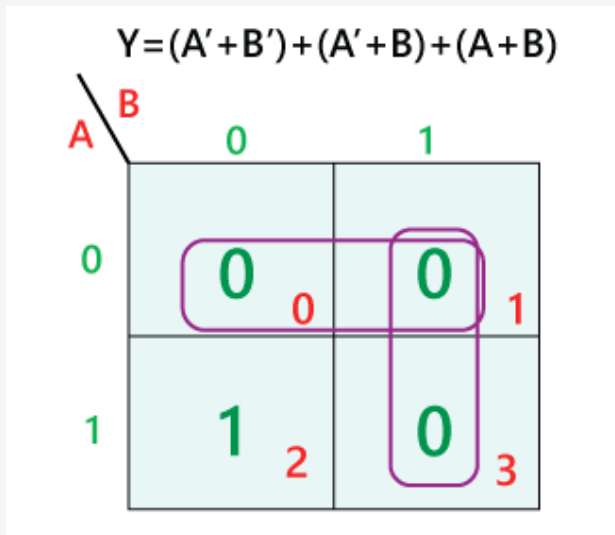
Maxterm Solution of K-Map (POS)

- ❖ find the simplified maxterm solution using K-map is the same as to find for the minterm solution. There are some minor changes in the maxterm solution, which are as follows:
 1. We will populate the K-map by entering the value of 0 to each sum-term into the K-map cell and fill the remaining cells with one's.
 2. We will make the groups of 'zeros' not for 'ones'.
 3. Now, we will define the boolean expressions for each group as sum-terms.
 4. At last, to find the simplified boolean expression in the POS form, we will combine the sum-terms of all individual groups.

Cont..

Example 1: $Y = (A' + B') + (A' + B) + (A + B)$

The max term product of $Y = \prod M(0, 1, 3)$



Simplified expression: $A'B$

Example 2

- ❖ Let us simplify the following Boolean function,
 $f(X,Y,Z) = \prod M(0,1,2,4)$ using K-map.
- ❖ The given Boolean function is in product of Max terms form.
- ❖ It is having 3 variables X, Y & Z. So, we require 3 variable K-map.
- ❖ The given Max terms are M_0, M_1, M_2 & M_4 .
- ❖ The 3 variable K-map with zeroes corresponding to the given Max terms is shown in the following figure.

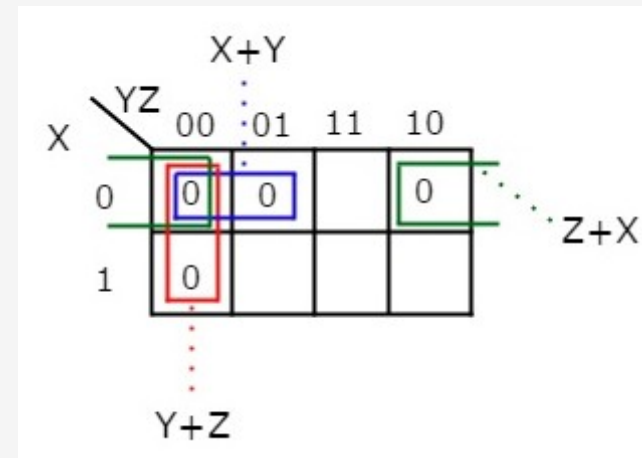
		YZ			
		00	01	11	10
X	0	0	0		0
	1	0			

cont..

- ❖ There are no possibilities of grouping either 8 adjacent zeroes or 4 adjacent zeroes. There are three possibilities of grouping 2 adjacent zeroes.
- ❖ After these three groupings, there is no single zero left as ungrouped.
- ❖ The 3 variable K-map with these three groupings is shown in the following figure.

❖ simplified Boolean function is

$$f = X + Y \cdot Y + Z \cdot Z + X$$



Lecture 3

Logic circuits

❖ Digital logic chips are combined to give us useful circuits.

These logic circuits can be categorized as either

- ✓ **Combinational logic**
- ✓ **Sequential logic.**

Combinational circuits

❖ Combinational circuit is a circuit in which we combine the different gates in the circuit.

❖ **Basic Concepts**

❖ The **output is always based on the given inputs.**

❖ The output of a combinational circuit is a function of its inputs, and the output is uniquely determined by the values of the inputs at **any given moment.**

Cont..

- ❖ The combinational circuit ***do not use any memory.***
- ❖ A given combinational circuit may have several outputs. If so, each output represents a **different** Boolean function
- ❖ Examples of combinational circuits are: adder, subtractors, code converters, decoders, encoders, digital multiplexers, demultiplexers.

Cont..

What is adder ?

- ❖ an adder is a digital logic circuit in electronics that is extensively used for the addition of numbers.
- ❖ It used to calculate addresses and related activities and calculate table indices in the ALU.
- ❖ Adders are basically classified into two types:
 - ✓ Half Adder
 - ✓ Full Adder.

Half adder

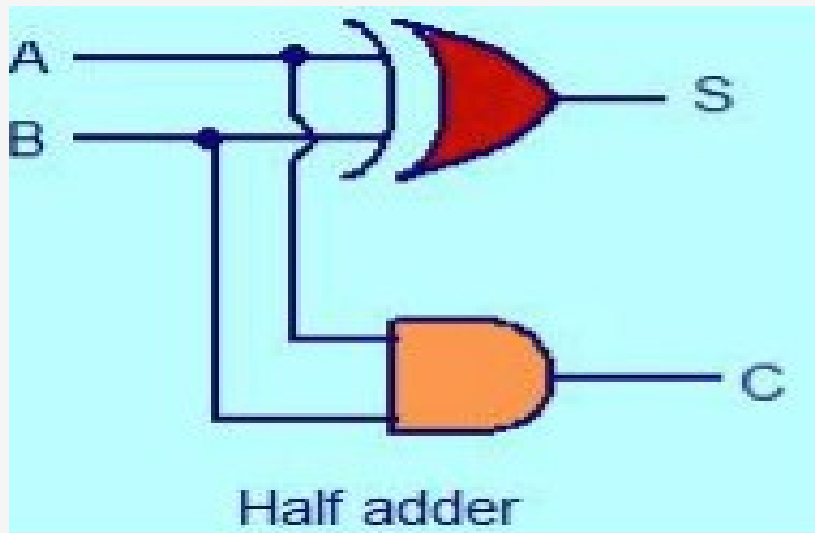
- ❖ Half adder is a combinational logic circuit with two inputs and two outputs.
- ❖ it is designed to add two single bit binary number A and B.
- ❖ This circuit has two outputs **carry** and **sum**.
 - ✓ The output obtained from the EX-OR gate is the **sum**
 - ✓ The output obtained from AND gate is the **carry**.

Cont.

❖ Logical symbol and truth table half adder

Logical Expression :

$$\text{sum} = \bar{a}.b + a.\bar{b} = a \oplus b$$
$$c_{out} = a.b$$



Truth Table			
Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Full Adder

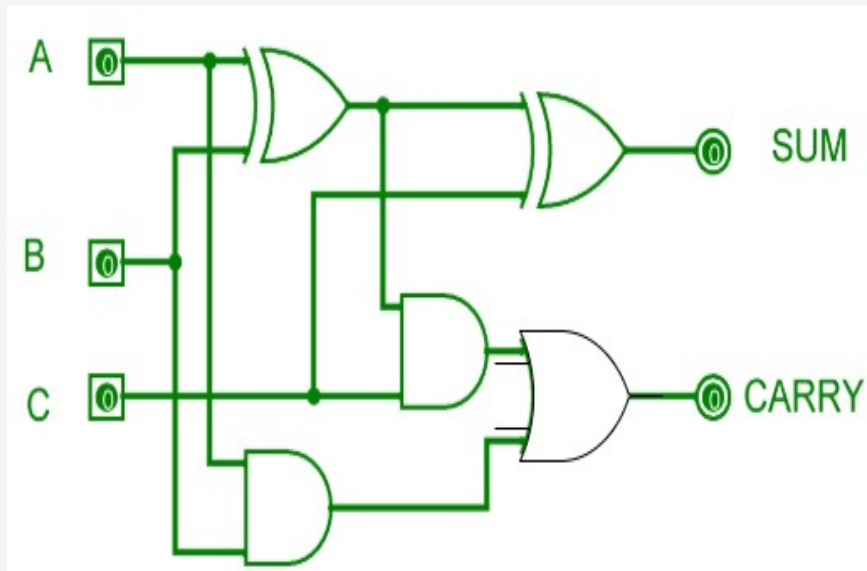
- ❖ A full adder logic circuit has three inputs A, B and C, which add the three input numbers and generate a **carry** and **sum**.
- ❖ The full adder adds 3 one bit numbers, where two can be referred to as operand and one can be referred to as bit carried in.
- ❖ it produces 2-bit output, and these can be referred to as output carry and sum.
- ❖ The difference between a half-adder and a fulladder is that the full-adder has three inputs and two outputs, whereas half adder has only two inputs and two outputs.

Cont..

- ❖ When full-adder logic is designed, you string eight of them together to create a byte-wide adder and cascade the carry bit from one adder to the next
- ❖ Full Adder is the circuit which consists of two EX-OR gates, two AND gates and one OR gate.
- ❖ The output carry is designated *as C-OUT* and the normal output is designated as *S which is SUM*.
- ❖ Sum output S is an XOR between the input A and the half-adder, SUM output with B and C-IN inputs.
- ❖ The values of C-OUT will only be true if any of the *two inputs out of the three are HIGH*.

Cont..

❖ Logical diagram and truth table of full adder



Input			Output	
A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

❖ Reading assignment

✓ Boolean expression for Sum and carry for full adder

$$\begin{aligned} S &= x'y'z + x'yz' + xy'z' + xyz \\ &= x \oplus y \oplus z \end{aligned}$$

$$\begin{aligned} C &= xy + xz + yz \\ &= xy + (x'y + xy')z \end{aligned}$$

Sequential Circuits

- ❖ The major **weakness** of **combinational circuits** is that there is no concept of storage – they are **memoryless**.
- ❖ If we change an input value, this has a direct and **immediate** impact on the value of the output.

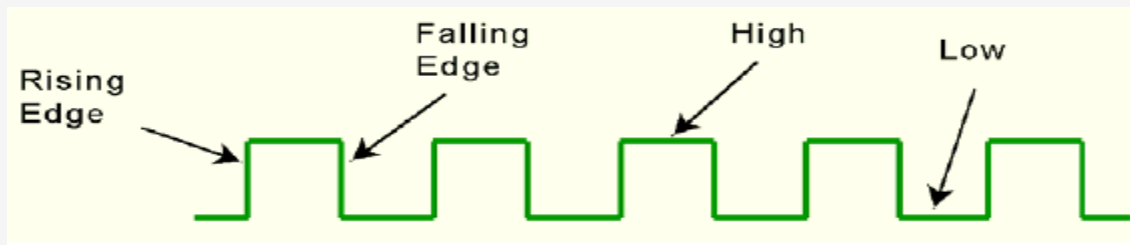
□ Basic Concepts

- ✓ A sequential circuit defines its output as a function of both its current inputs and its previous inputs. Therefore, the output depends on **past** inputs.
- ✓ We typically refer to this storage element as a **flip-flop**.

Cont..

❑ Clocks

- ❖ A sequential circuit uses **past inputs** to determine **present outputs** indicates we must have event ordering.
- ❖ A clock is a circuit that emits a series of pulses with a precise pulse width and a precise **interval** between consecutive pulses.
- ❖ This interval is called the **clock cycle time**. Clock speed is generally measured in megahertz (MHz),
- ❖ A clock is used by a **sequential circuit** to decide **when** to update the state of the circuit.



Cont..

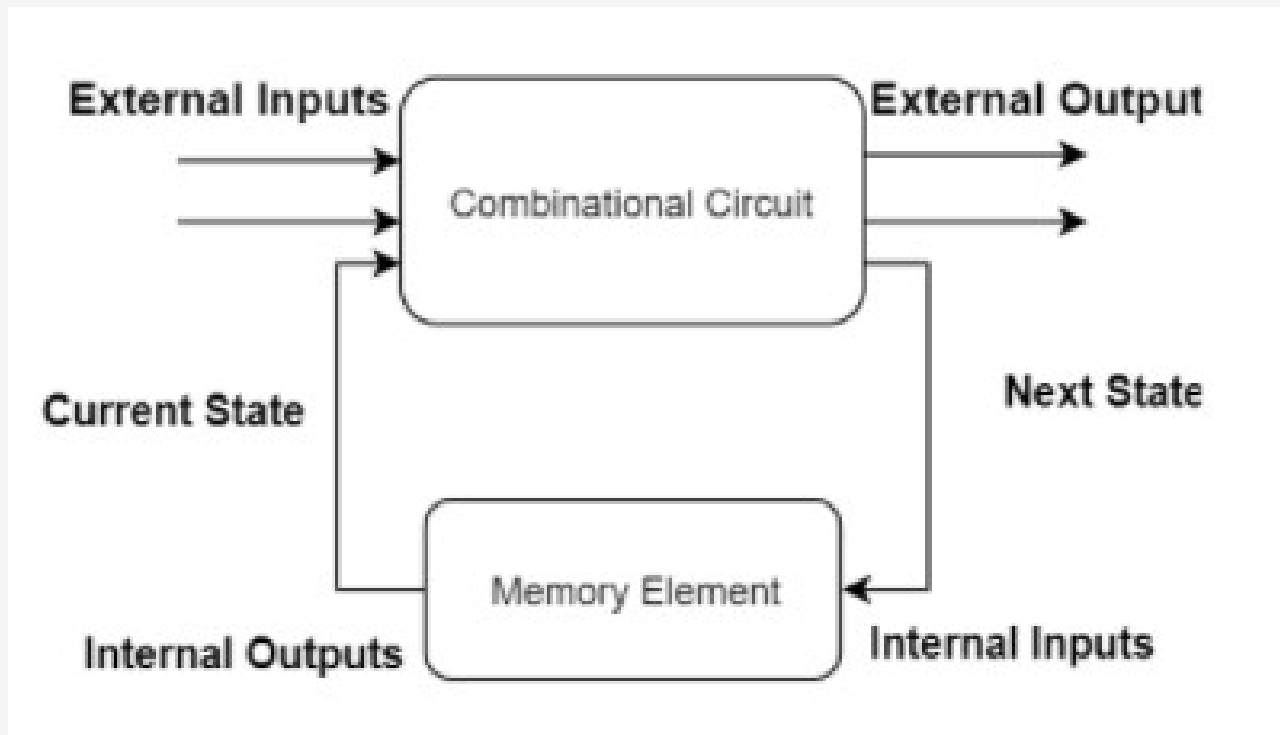
- ❖ A sequential circuit is the *assimilation of a combinational logic circuit and a storage element*.
- ❖ With the applied inputs to the combinational logic, the circuit outputs are derived.
- ❖ These sequential circuits deliver the *output based on both the current and previously stored* input variables.
- ❖ The derived output is passed on to the next clock cycle.
- ❖ Sequential circuits consist of memory *devices to store binary data*.

Cont..

- ❖ This binary information describes the current state of the sequential circuit.
- ❖ sequential circuits work along with the combinational circuit, there are two types of combinational logic inputs where those are
 - ✓ *External inputs* where these are not monitored by the circuit
 - ✓ *Internal inputs* are the function of previous state output.

Cont..

basic sequential circuit



Block diagram for sequential circuit

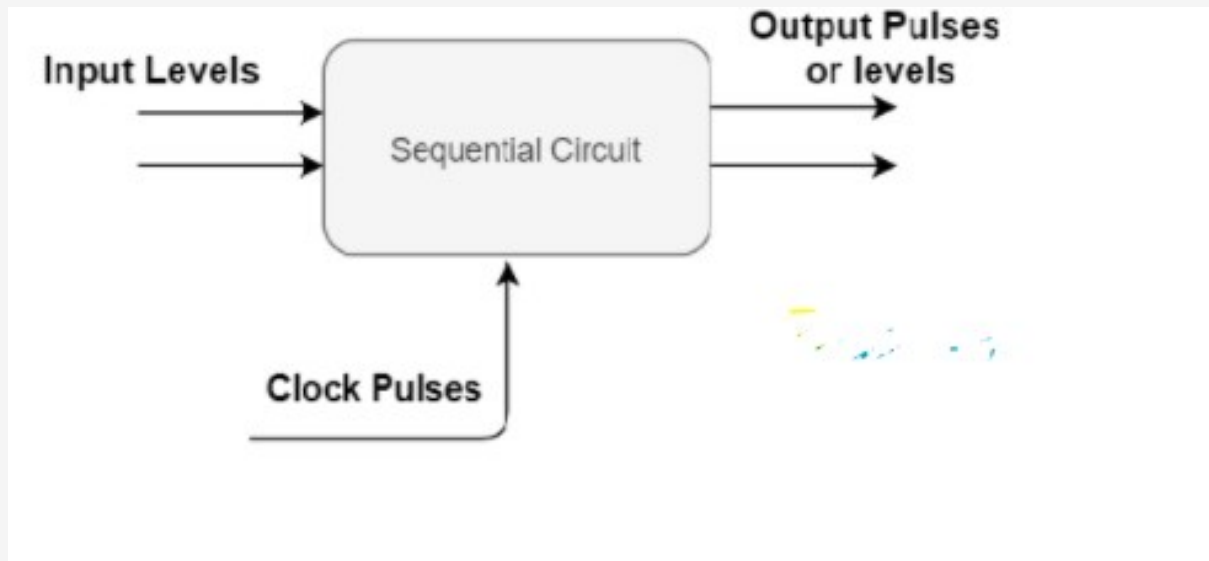
Types of Sequential Circuits

Synchronous Sequential Circuits:-

- ❖ Synchronous sequential circuits use level inputs and clock signals as the circuit inputs
- ❖ The duration of the output pulse is like the clock pulse of the clocked circuits.
- ❖ Level output modified its state at the beginning of an input pulse and continues in that until the next clock pulse arrives.

Cont..

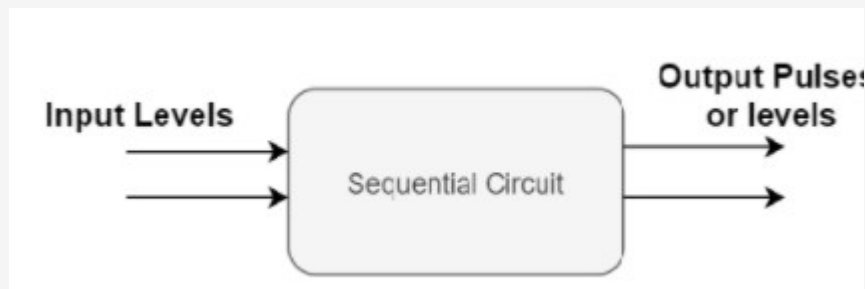
- ❖ Until the arrival of the next pulse, these circuits perform no activity and so the operation is a bit slower than that of asynchronous circuits



Cont..

Asynchronous Sequential Circuits

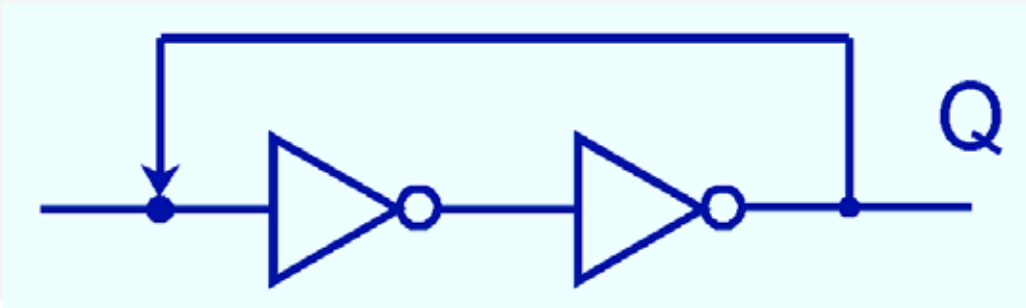
- ❖ Asynchronous sequential circuits perform their operation without depending on the clock signal but use the input pulses and generate the output.
- ❖ As there is no clock pulse dependency, these circuits can switch to the next state quickly when the input signal is changed. So, there is a faster operation with asynchronous sequential circuit



Cont.

Flip-Flops

- ❖ Many people use the terms **latch** and flip-flop interchangeably. Technically, a latch is level triggered, whereas a flip-flop is edge triggered.
- ❖ In order to “remember” a past state, sequential circuits rely on a concept called **feedback**. This simply means the output of a circuit is fed back as an input to the same circuit

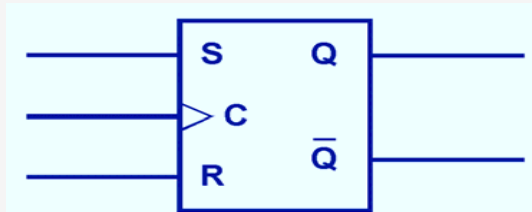


Cont..

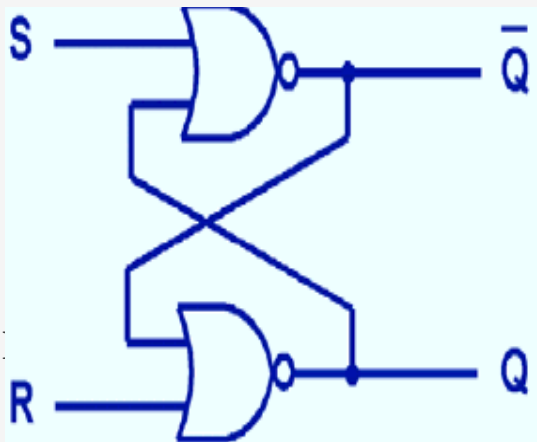
- ❖ Flip flops are essential in data storage.
- ❖ They are electronic circuits with two stable states used to store binary data. Such a circuit has one or more control inputs and one or two outputs.
- ❖ By applying varying input, the data stored can be changed.
- ❖ In sequential logic, the flip flop is the basic storage element.
- ❖ They are fundamental building blocks of electronics systems such as computers and communication device

Types Of Flip Flops

- ❖ A more useful feedback circuit is composed of two NOR gates resulting in the most basic memory unit call an **SR flip-flop**. SR stands for “**set/reset**.”



- ❖ $Q(t)$ means the value of the output at time t . $Q(t+1)$ is the value of Q after the **next** clock pulse.
- ❖ When both S and R are 1, the SR flip-flop is **unstable**.

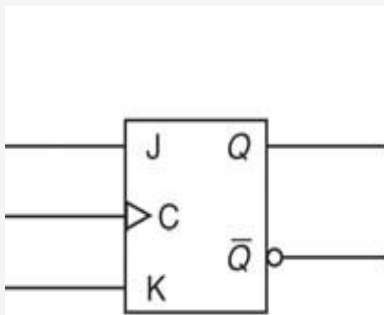


The Actual S

S	R	$Q(t+1)$
0	0	$Q(t)$ (no change)
0	1	0 (reset to 0)
1	0	1 (set to 1)
1	1	undefined

Cont..

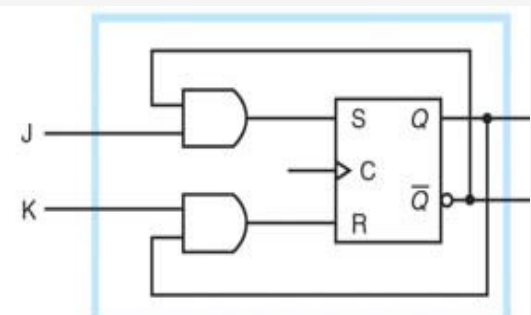
- ❖ A **JK flip-flop** simply modify the SR flip-flop to ensure that the **illegal state** (both S and R are 1) never arises.
- ❖ The inputs to an SR flip-flop will **never** both be 1
- ❖ We will never have an **unstable** circuit.



(a)

J	K	$Q(t+1)$
0	0	$Q(t)$ (no change)
0	1	0 (reset to 0)
1	0	1 (set to 1)
1	1	$\bar{Q}(t)$

(b)

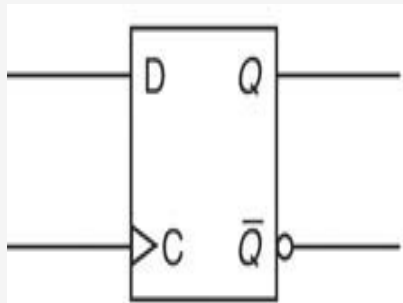


(c)

A JK Flip-Flop; b) The JK Characteristic Table c) A JK Flip-Flop as a Modified SR Flip-Flop

Cont..

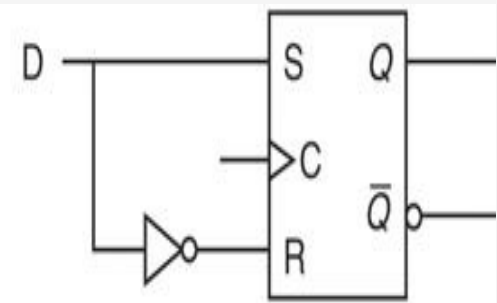
- ❖ A **D flip-flop** is a **true** representation of physical computer memory.
- ❖ An output value of 1 means the circuit is currently “**storing**” a value of 1.



(a)

D	Q (t+1)
0	0
1	1

(b)



(c)

- ❖ *A D Flip-Flop; b) The D Characteristic Table c) A D Flip-Flop as a Modified SR Flip-Flop*

Excitation Tables

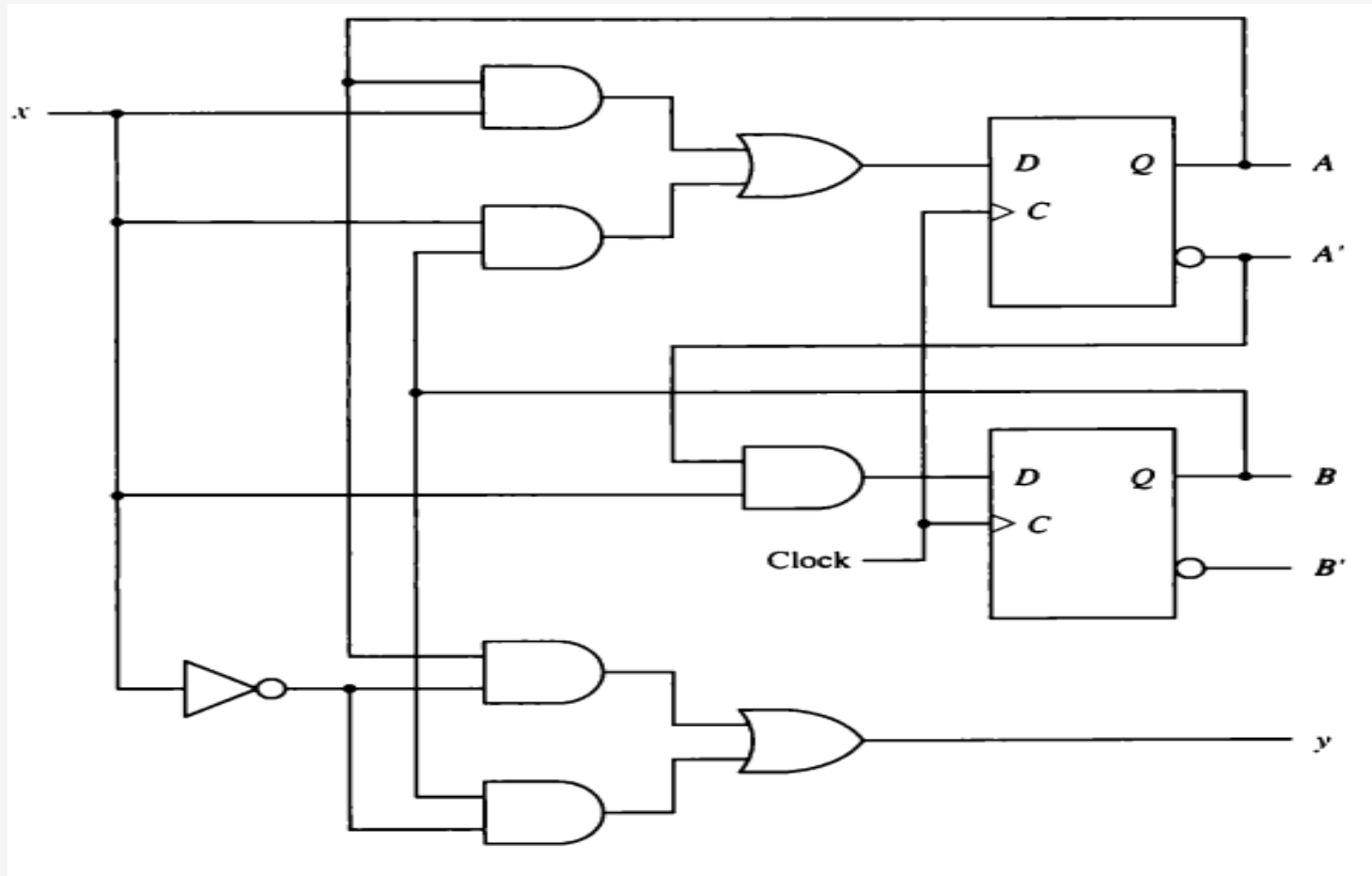
- The characteristic tables of flip-flops specify the next state when the inputs and the present state are known.
- During the design of sequential circuits we usually know the required transition from present state to next state and wish to find the flip-flop input conditions that will cause the required transition.
- For this reason we need a table that lists the required input combinations for a given change of state. Such a table is called a flip-flop excitation table.

SR flip-flop				D flip-flop		
$Q(t)$	$Q(t + 1)$	S	R	$Q(t)$	$Q(t + 1)$	D
0	0	0	x	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	0
1	1	x	0	1	1	1

JK flip-flop				T flip-flop		
$Q(t)$	$Q(t + 1)$	J	K	$Q(t)$	$Q(t + 1)$	T
0	0	0	x	0	0	0
0	1	1	x	0	1	1
1	0	x	1	1	0	1
1	1	x	0	1	1	0

$Q(t)$	J	K	$Q(t+1)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Flip-Flop Input Equations



Example of sequential circuit

-
- From the figure, it has one input variable x , one output variable y , and two clocked D flip-flops. The AND gates, OR gates, and inverter form the combinational logic part of the circuit.
 - The interconnections among the gates in the combinational circuit can be specified by a set of Boolean expressions. The part of the combinational circuit that generates the inputs to flip-flops are described by a set of Boolean expressions called flip-flop input equations. We adopt the convention of using the flip-flop input symbol to denote the input equation variable name and a subscript to designate the symbol chosen for the output of the flip-flop.
 - we have two input equations, designated DA and DB . The first letter in each symbol denotes the D input of a D flip-flop. The subscript letter is the symbol name of the flip-flop.

Con't

- Since the output of the OR gate is connected to the D input of flip-flop A, we write the first input equation as:

$$D_A = A x + B x$$

- The second input equation is:

$$D_B = A' x$$

- The external output equation is expressed as:

$$y = A x' + B x'$$

State table

- The behavior of a sequential circuit is determined from the inputs, the outputs, and the state of its flip-flops. Both the outputs and the next state are a function of the inputs and the present state.
- A sequential circuit is specified by a state table that relates outputs and next states as a function of inputs and present states. In clocked sequential circuits, the transition from present state to next state is activated by the presence of a clock signal.
- The table consists of four sections, labeled present state, input, next state, and output
- The present-state section shows the states of flip-flops A and B at any given time t .
- The input section gives a value of x for each possible present state.
- The next-state section shows the states of the flip-flops one clock period later at time $t + 1$.
- The output section gives the value of y for each present state and input condition.

Cont

- The next-state value and output are determined from the figure or using the input functions.

State table of the above figure

Present state		Input	Next state		Output
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

$$DA = Ax + Bx$$

$$DB = A'x$$

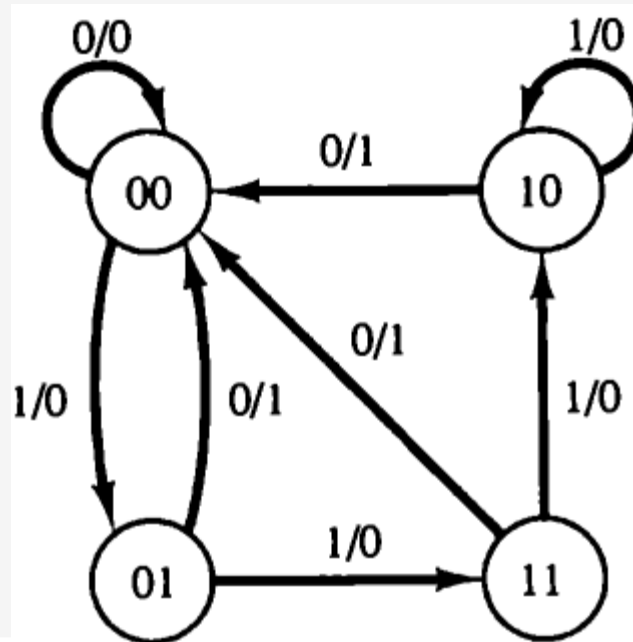
$$y = Ax' + Bx'$$

State diagram

- The information available in a state table can be represented graphically in a state diagram.
- In this type of diagram, a state is represented by a circle, and the transition between states is indicated by directed lines connecting the circles.
- The state diagram provides the same information as the state table and is obtained directly from state table.
- The binary number inside each circle identifies the state of the flip-flops.
- The directed lines are labeled with two binary numbers separated by a slash. The input value during the present state is labeled first and the number after the slash gives the output during the present state.
- For example, the directed line from state 00 to 01 is labeled 1/0, meaning that when the sequential circuit is in the present state 00 and the input is 1, the output is 0. After a clock transition, the circuit goes to the next state 01.

Cont

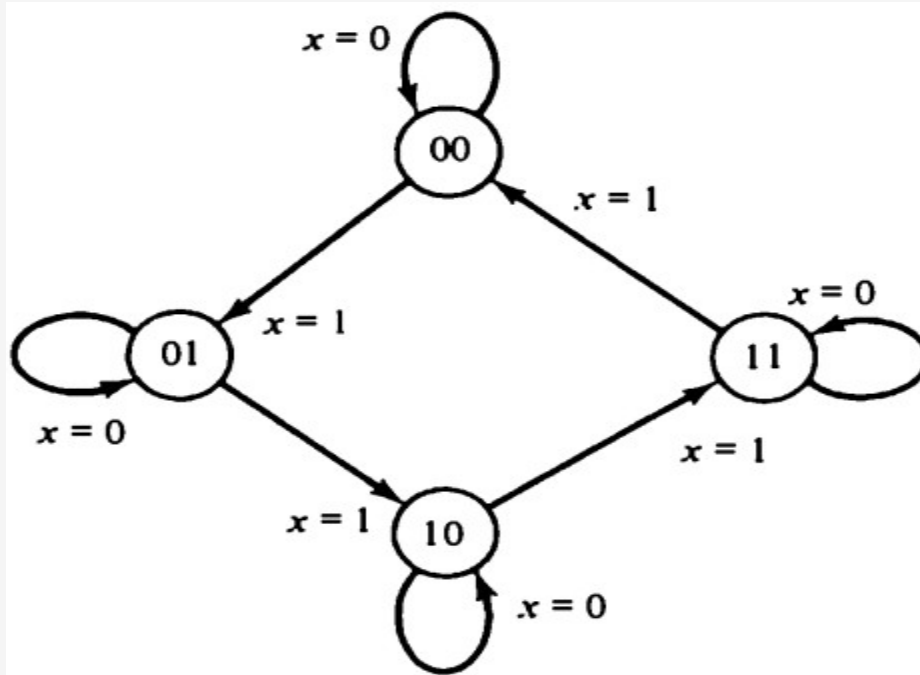
- The state table is easier to derive from a given logic diagram and the state diagram follows directly from the state table. The state diagram gives a pictorial view of state transitions and is the form suitable for human interpretation of the circuit operation.



State diagram of the circuit

Design example

- We wish to design a clocked sequential circuit that goes through a sequence of repeated binary states 00, 01, 10, and 11 when an external input x is equal to 1. The state of the circuit remains unchanged when $x = 0$.



- This sequential circuit has no external outputs, and therefore only the input value is labeled in the diagram

Cont

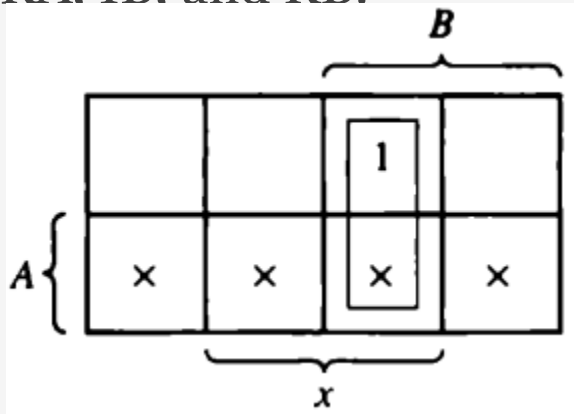
- The excitation table of a sequential circuit is an extension of the state table. This extension consists of a list of flip-flop input excitations that will cause the

Excitation table

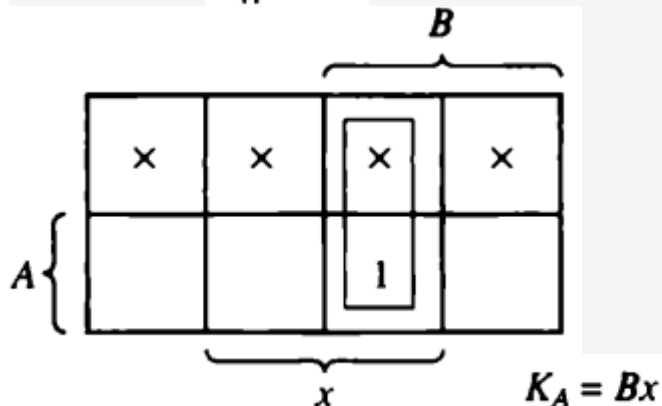
Present state		Input	Next state		Flip-flop inputs			
A	B		A	B	J_A	K_A	J_B	K_B
0	0	0	0	0	0	×	0	×
0	0	1	0	1	0	×	1	×
0	1	0	0	1	0	×	×	0
0	1	1	1	0	1	×	×	1
1	0	0	1	0	×	0	0	×
1	0	1	1	1	×	0	1	×
1	1	0	1	1	×	0	×	0
1	1	1	0	0	×	1	×	1

Cont

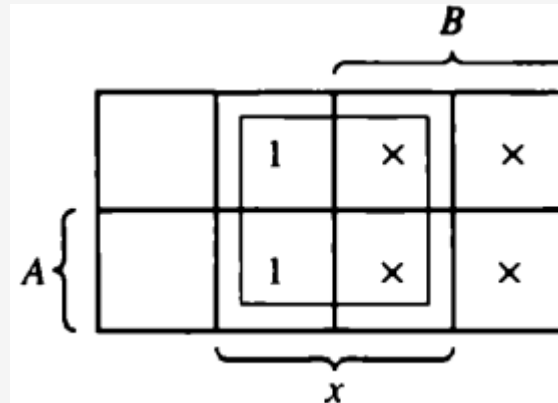
The inputs are the variables A, B, and x. The outputs are the variables IA , KA , IB , and KB .



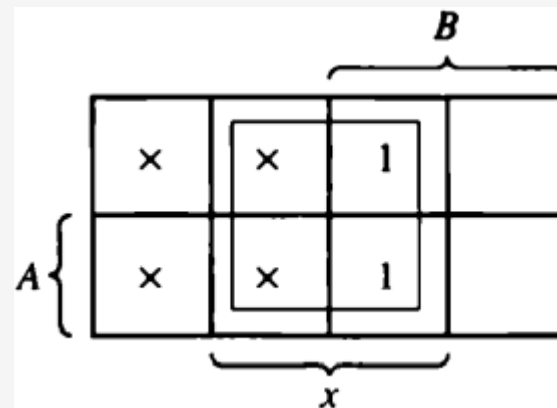
$$J_A = Bx$$



$$K_A = Bx$$



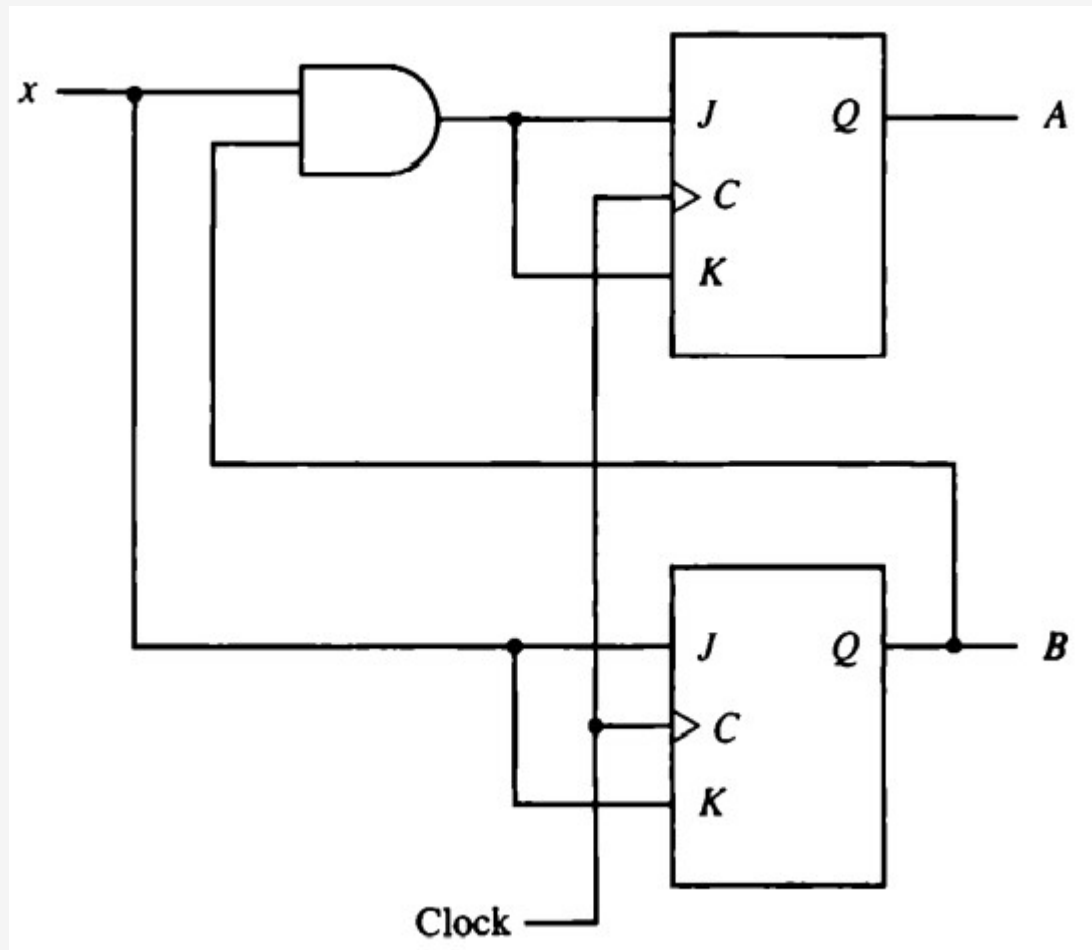
$$J_B = x$$



$$K_B = x$$

Cont

Finally, map of the sequential circuit will be:



$$J_A = Bx$$

$$K_A = Bx$$

$$J_B = x$$

$$K_B = x$$

Thank you

?????